UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 6: Data Locality and Tiled Matrix Multiply

Volodymyr Kindratenko

2/5/2026

Course Reminders

- Lab 2 is due this Friday
  - Requires Labs 0 and 1 to be completed first
  - Included in the final grade!
- Reminder about Late Submission for Labs and Project:
  - **All assignments are due by Friday at 8pm.**
  - However, recognizing potential issues such as Delta system maintenance, interviews, or illness, we provide a **three-day** grace period. You can submit an assignment by Monday at 8pm and still receive full credit without penalty. You do not need to notify us if you intend to use this default 3-day grace period.
  - To extend your grace period beyond 3 days, you must notify us by Friday 8pm via email to the TA(s) responsible for the assignment you are asking about. Extensions to the grace period are given out **for extenuating circumstances at our discretion**. Requests after this time will not be considered.
  - **Because of this extension policy and because a lab with the lowest score will be dropped from the final grade, no late submissions will be allowed for any reasons for both labs and projects.**

Today's Objectives

- To prepare for Lab 3: tiled matrix multiplication
- To learn to handle boundary conditions in tiled algorithms
- To learn to assess the benefit of tiling

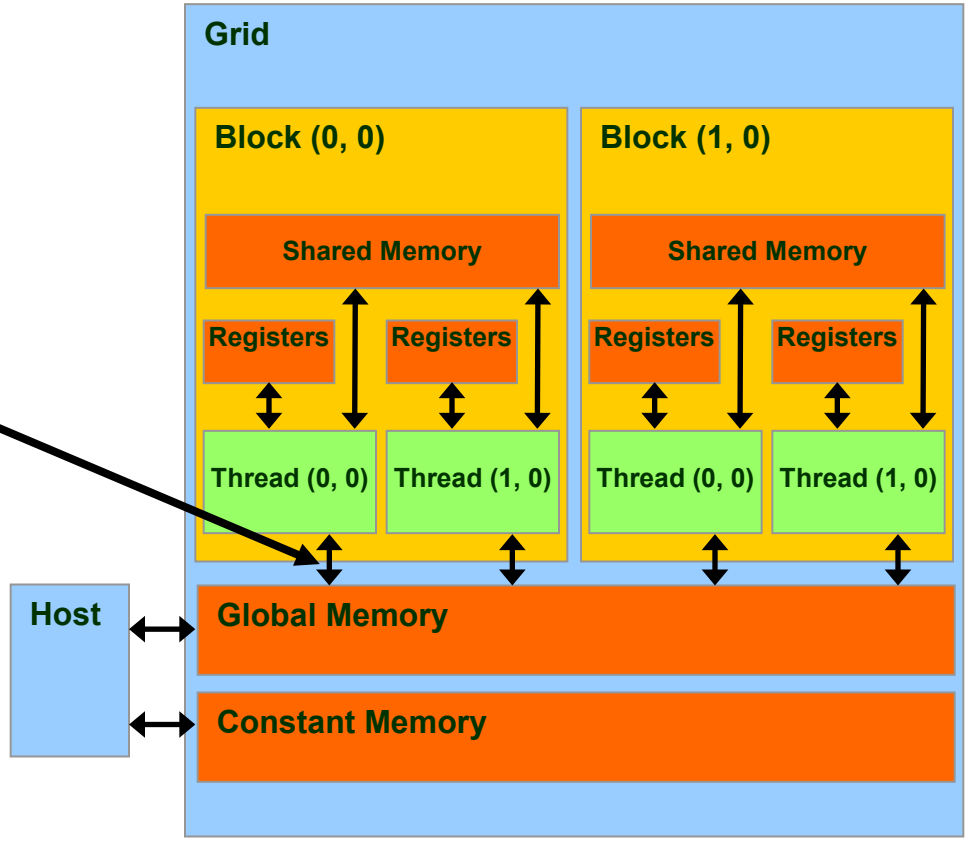
Review: 150 GB/s Bandwidth Implies 37.5 GFLOPs

- One generation of GPUs:
  - **1,000 GFLOP/s** of compute power, and
  - **150 GB/s** of memory bandwidth

- Dividing bandwidth by memory requirements:

$$\frac{150\ GB/s}{4\ B/FLOP} = 37.5\ \text{GFLOP/s}$$

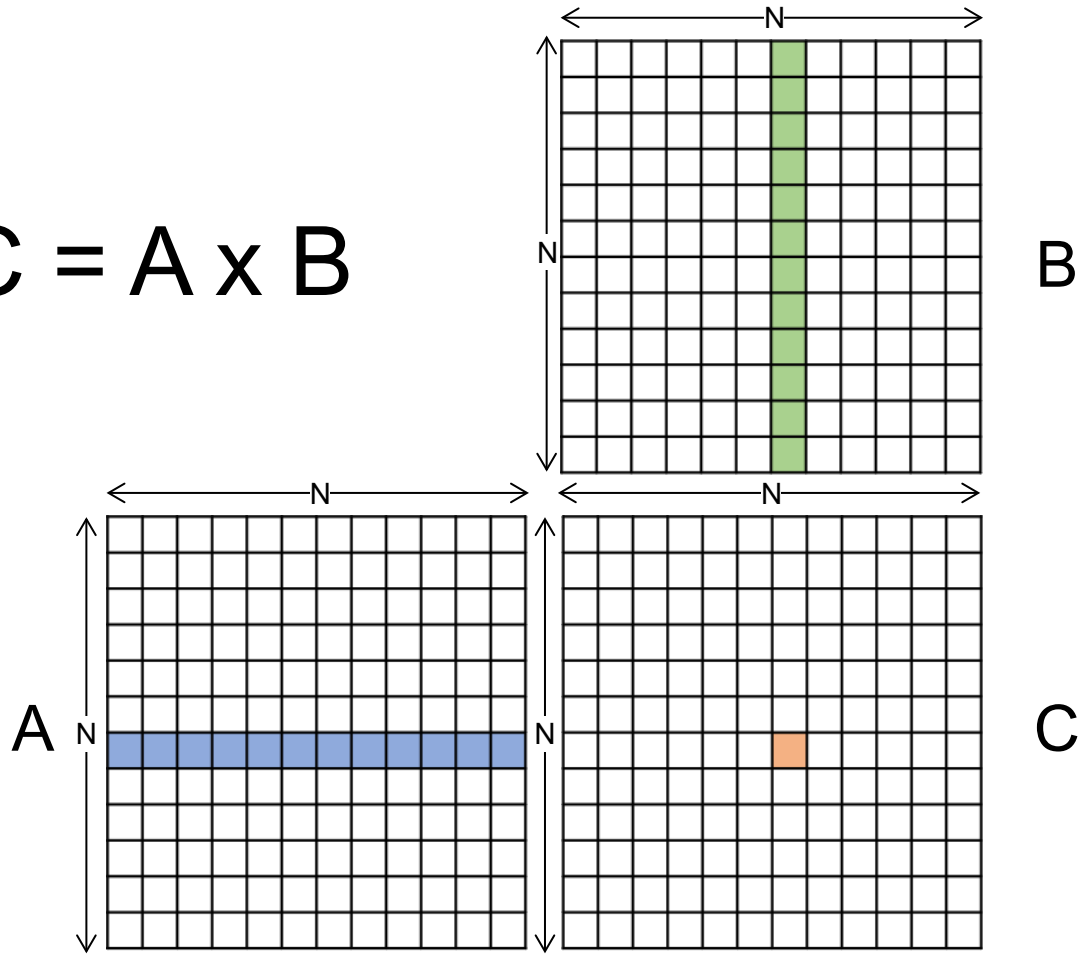
which **limits computation**!



Review: Matrix-Matrix Multiplication

- Given two N × N matrices, A and B,
  - we can multiply A by B
  - to compute a third N × N matrix, C: C = AB

C = A x B



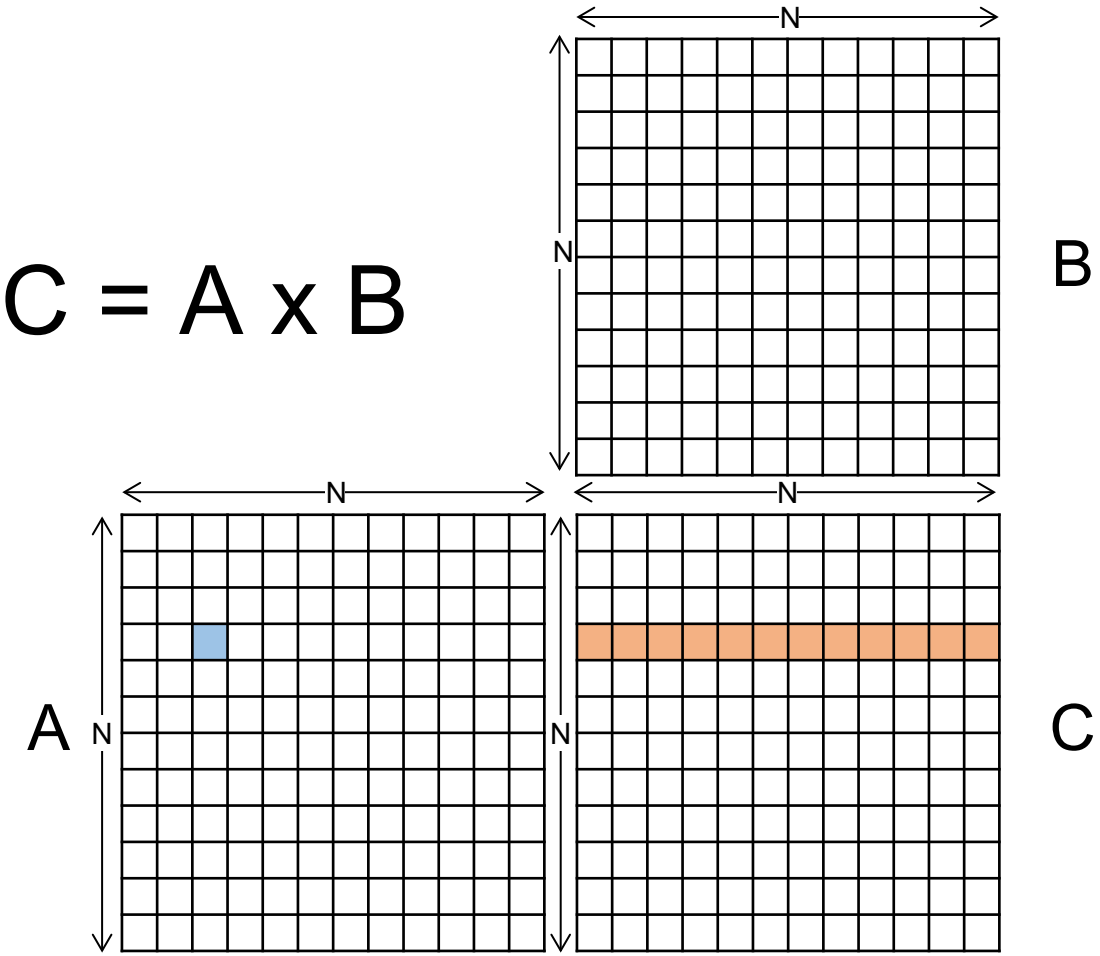
- In terms of the elements of C, matrix multiplication implies computing…

$$C_{ij} = \sum_{k=1}^{Width} A_{ik} B_{kj}$$

Reuse in Matrix-Matrix Multiplication

- Every element of the input matrix A is used for computing an entire row of the output matrix C

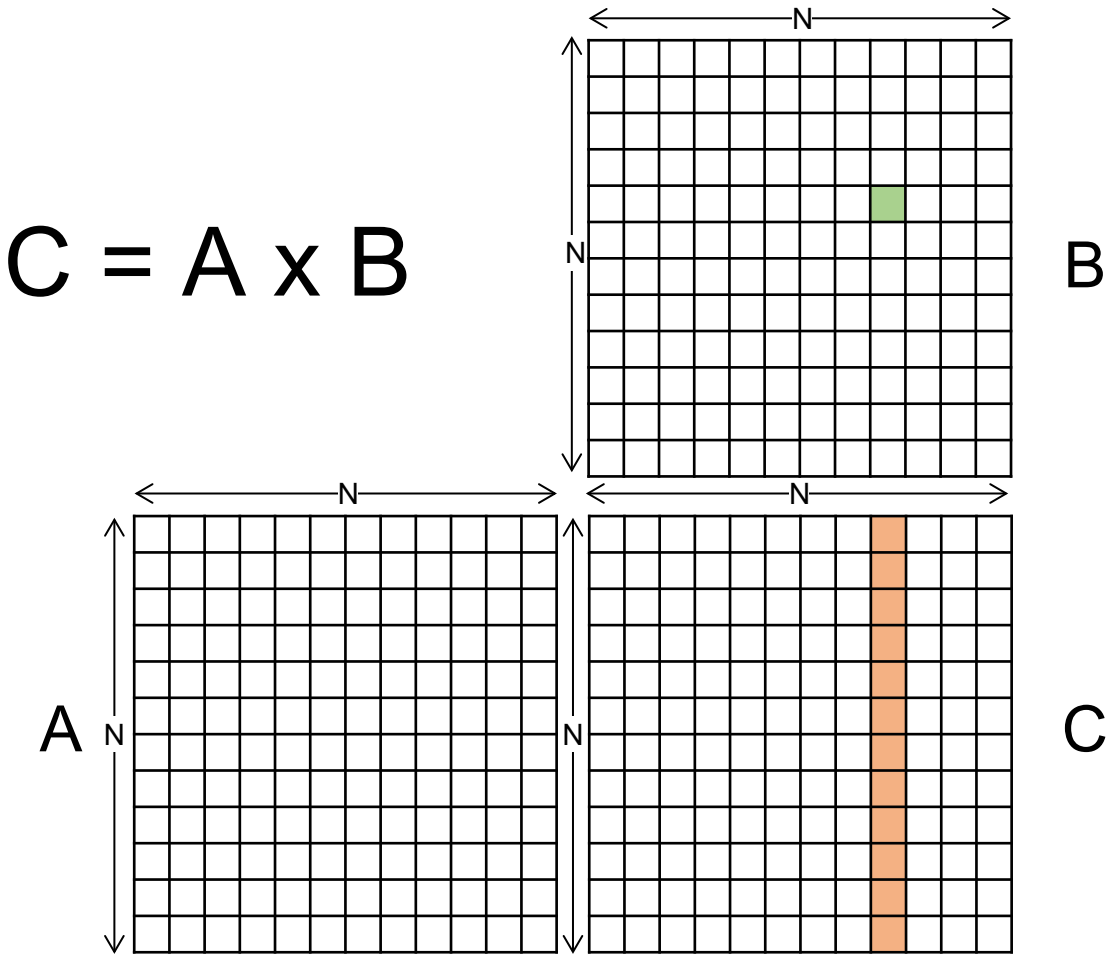
C = A x B



Reuse in Matrix-Matrix Multiplication

- Every element of the input matrix B is used for computing an entire column of the output matrix C

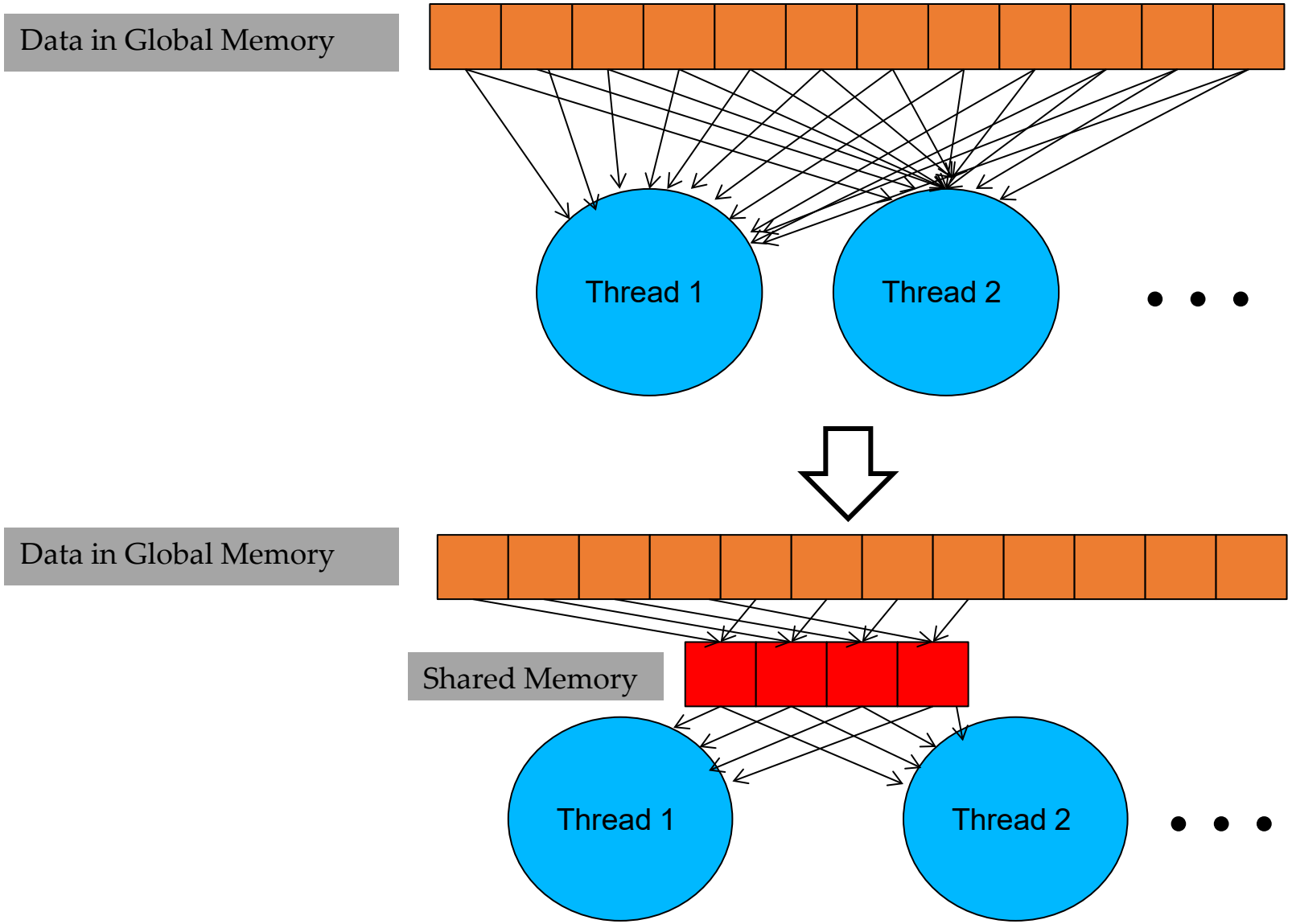
<u>Opportunity:</u> Optimize memory accesses in the kernel code to reuse input data loaded from global memory as much as possible



A Common Programming Strategy

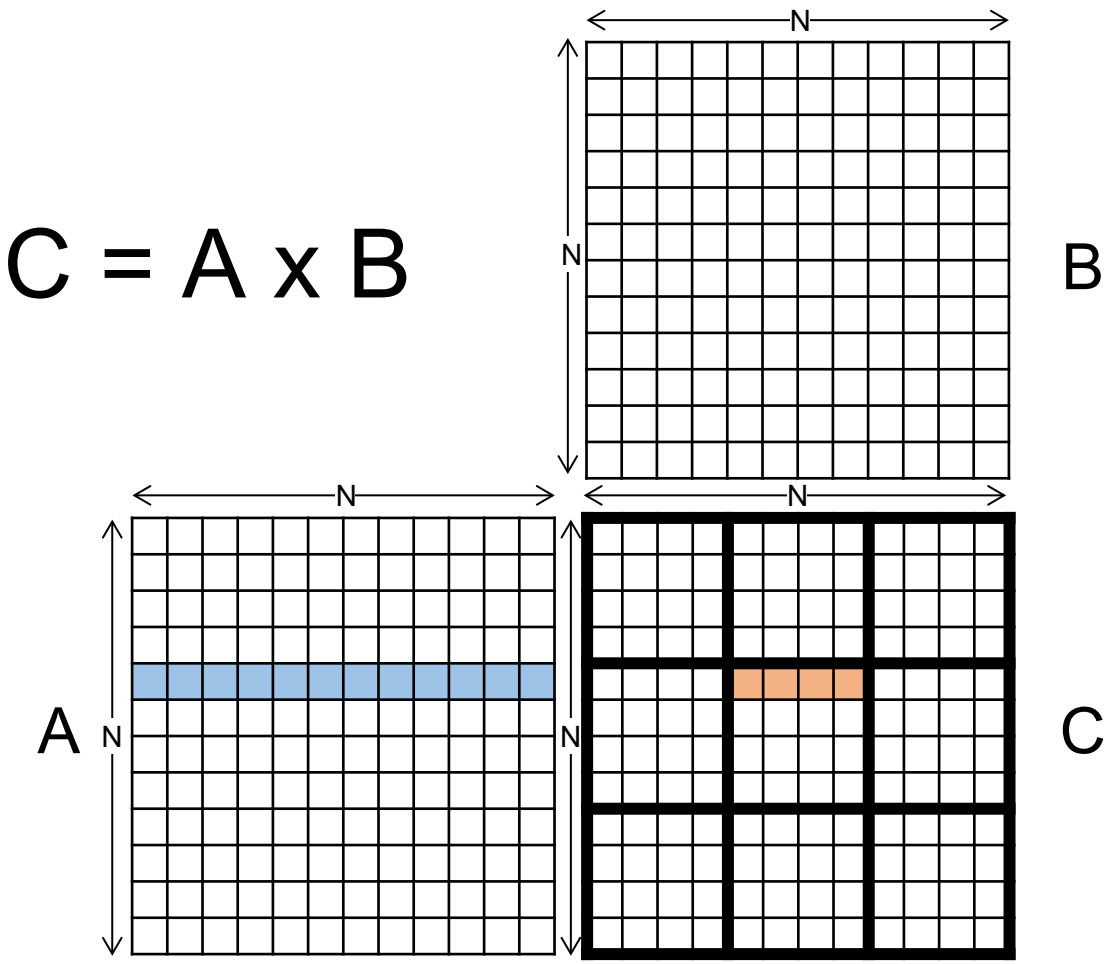
- Global memory is implemented with DRAM – slow
  - Sometimes, we are lucky:
    - The thread finds the data in the L1 cache because it was recently loaded by another thread
  - Sometimes, we are not lucky:
    - The data gets evicted from the L1 cache before another thread tries to load it
- To avoid Global Memory bottleneck, tile the input data to take advantage of Shared Memory:
  - Partition data into subsets (tiles) that fit into the (smaller but faster) shared memory
  - Handle each data subset with one thread block by:
    - Loading the subset from global memory to shared memory, using multiple threads to exploit memory-level parallelism
    - Performing the computation on the subset from shared memory; each thread can efficiently access any data element
    - Copying results from shared memory to global memory
  - Tiles are also called *blocks* in the literature

Shared Memory Tiling Basic Idea



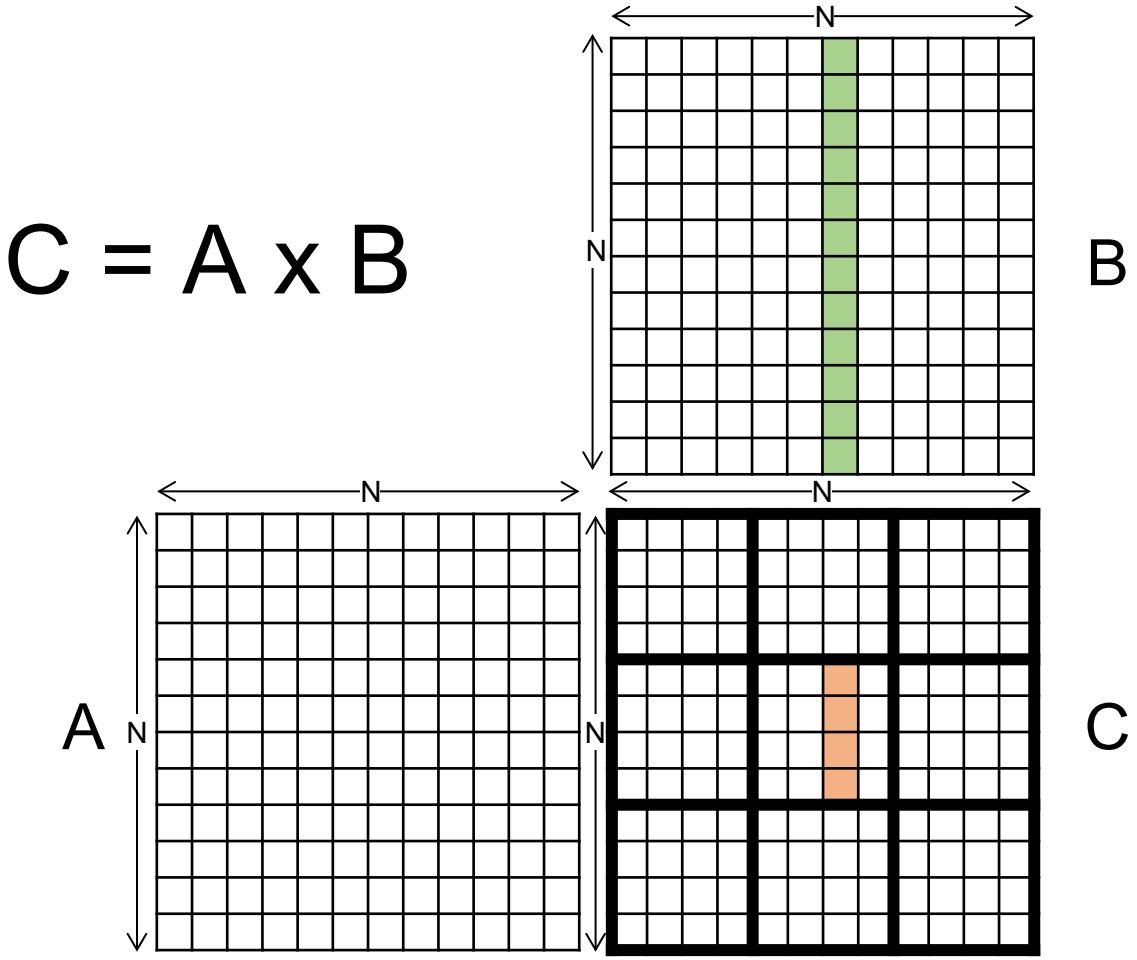
Reuse in Matrix-Matrix Multiplication

- Some of the threads in the same thread block use the same input data



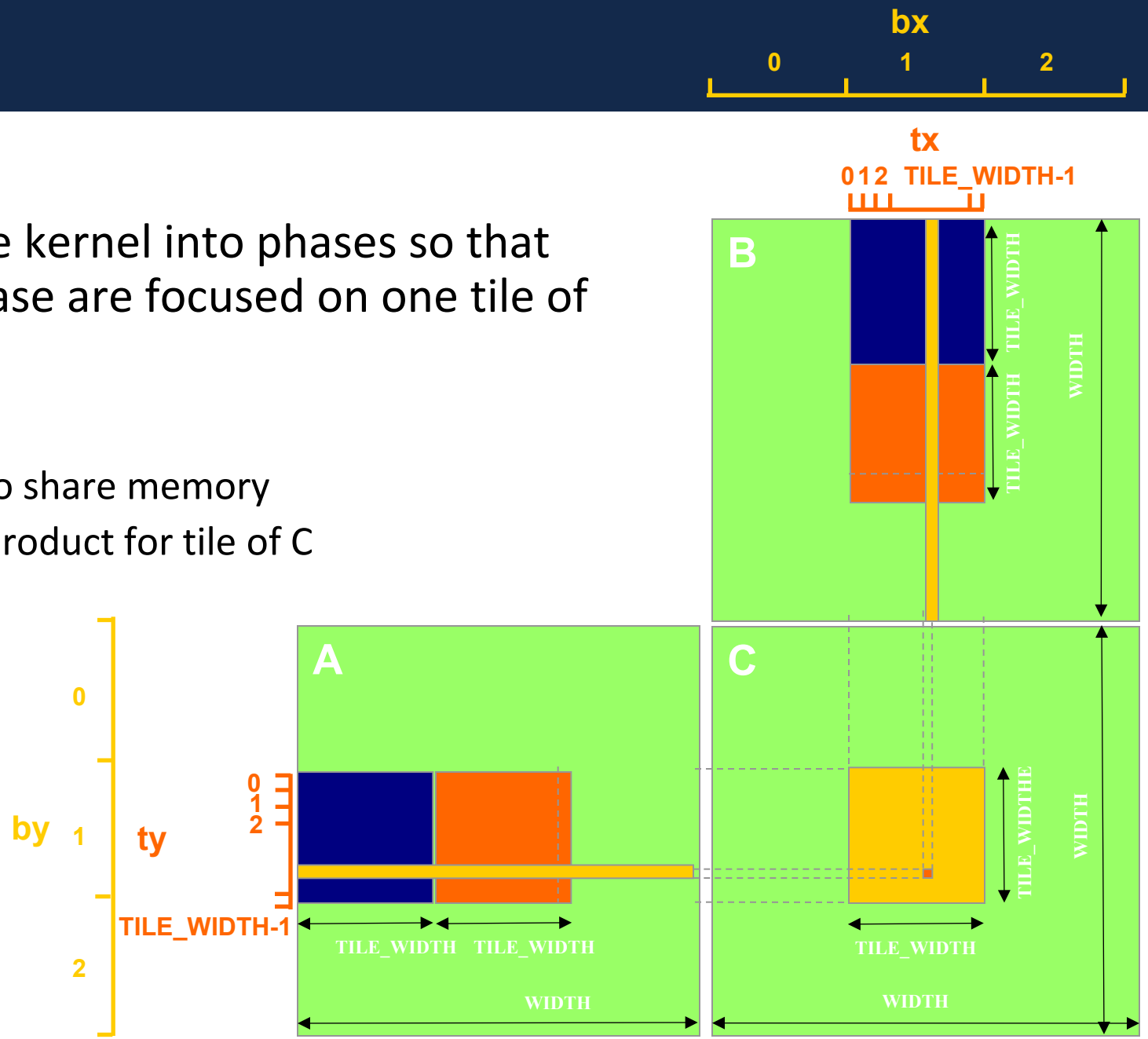
Reuse in Matrix-Matrix Multiplication

- Some of the threads in the same thread block use the same input data

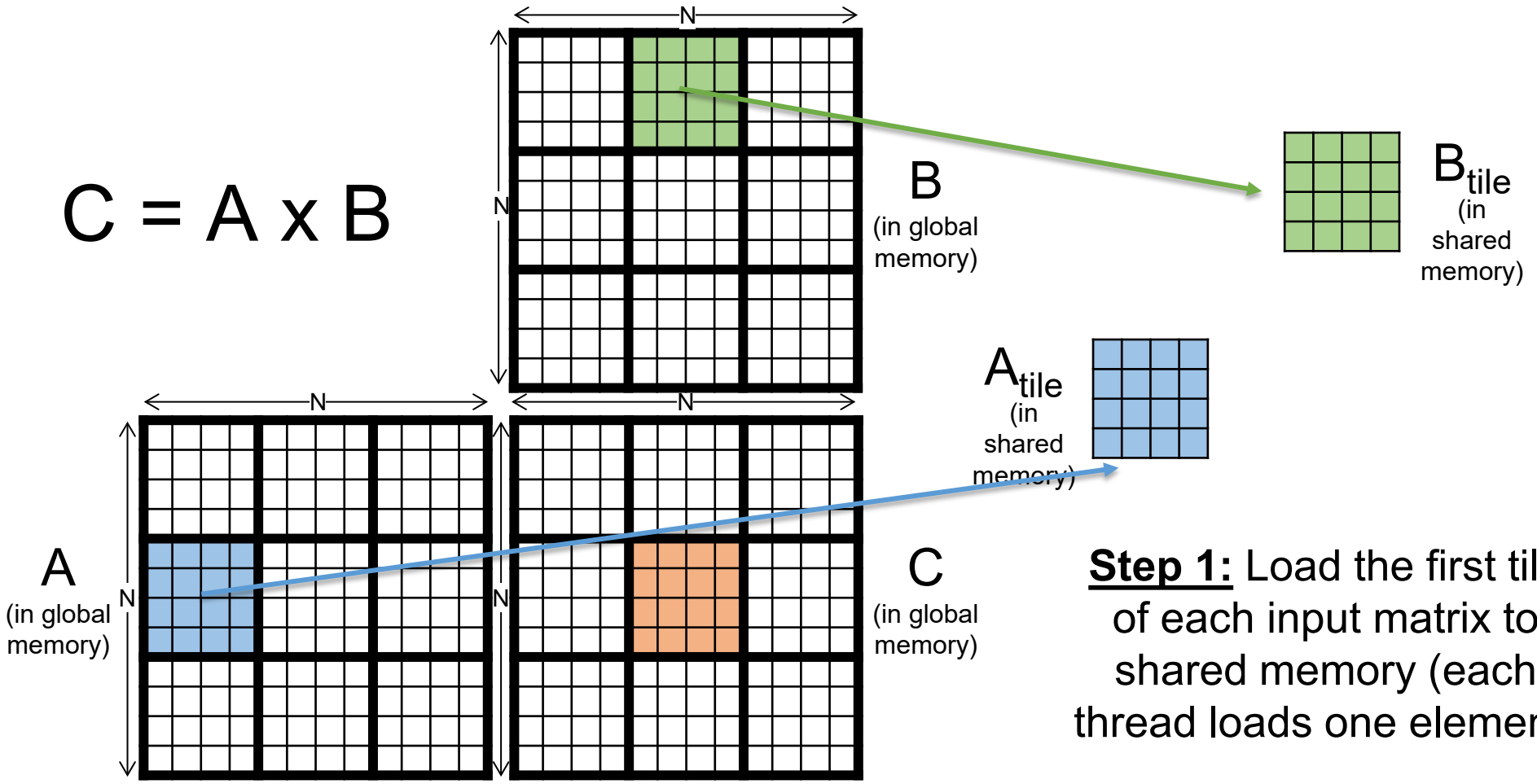


Tiled Multiply

- Break up the execution of the kernel into phases so that the data accesses in each phase are focused on one tile of A and B
- For each tile:
  - Phase 1: Load tiles of A & B into share memory
  - Phase 2: Calculate partial dot product for tile of C

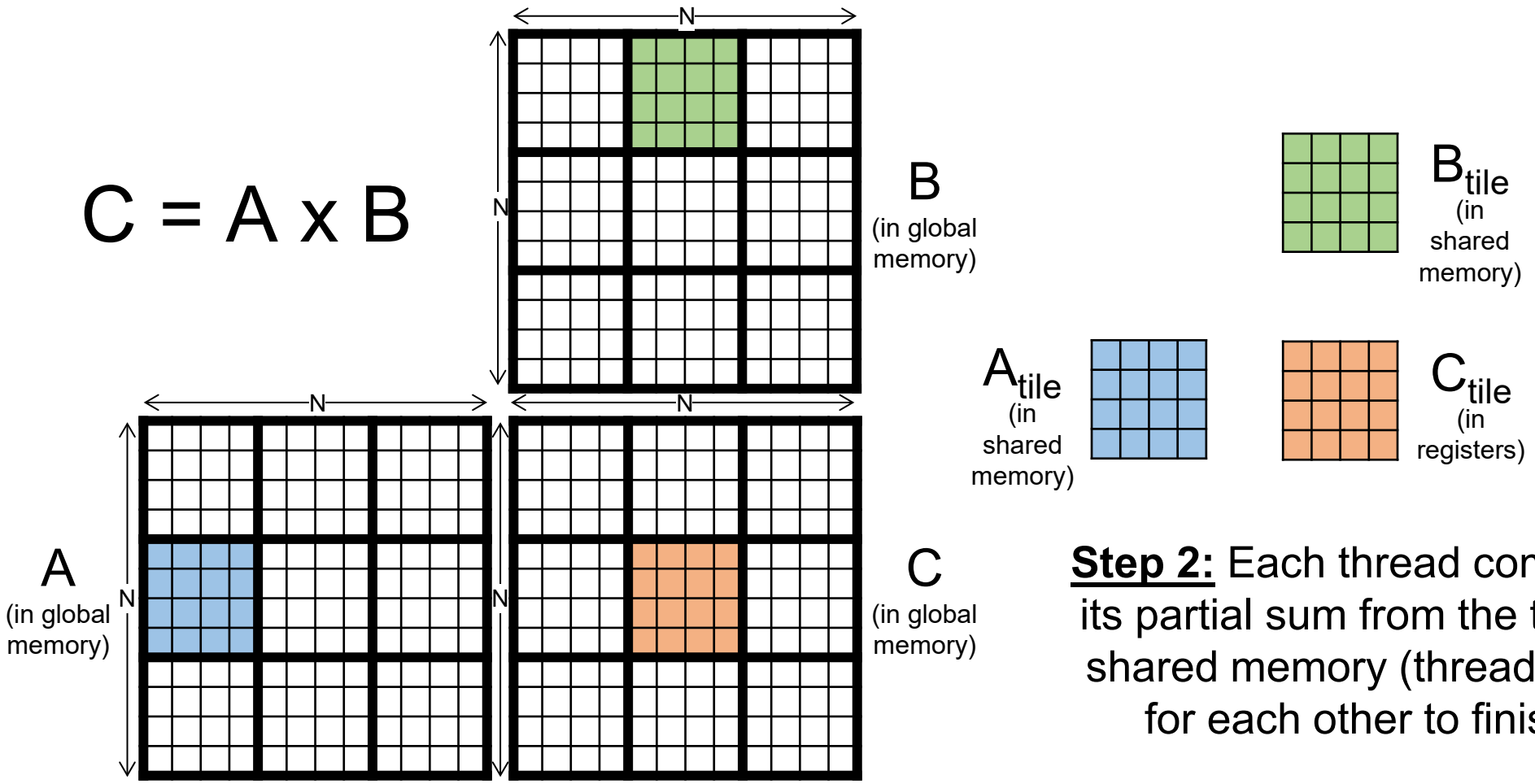


Tiled Matrix-Matrix Multiplication



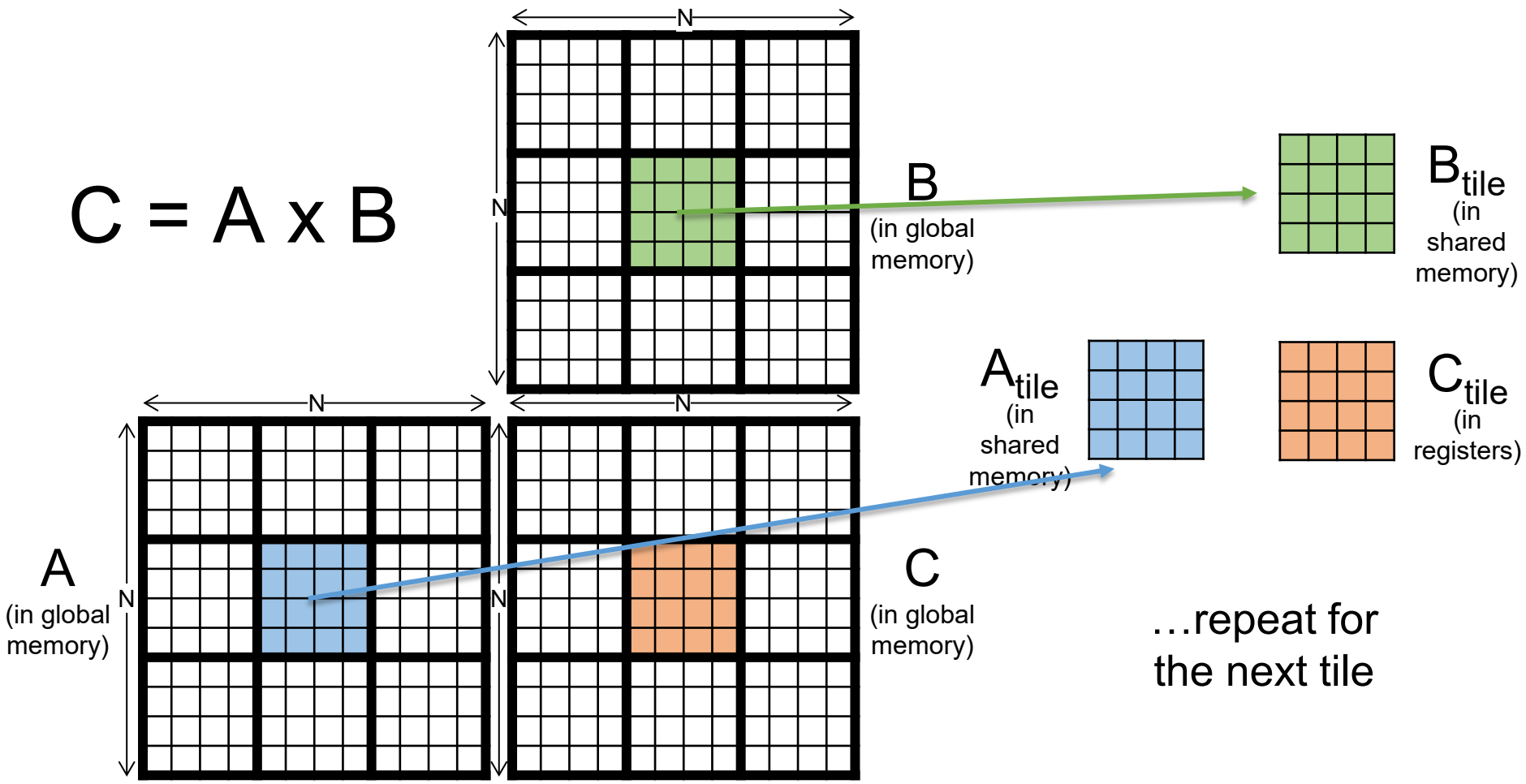
Step 1: Load the first tile of each input matrix to shared memory (each thread loads one element)

Tiled Matrix-Matrix Multiplication

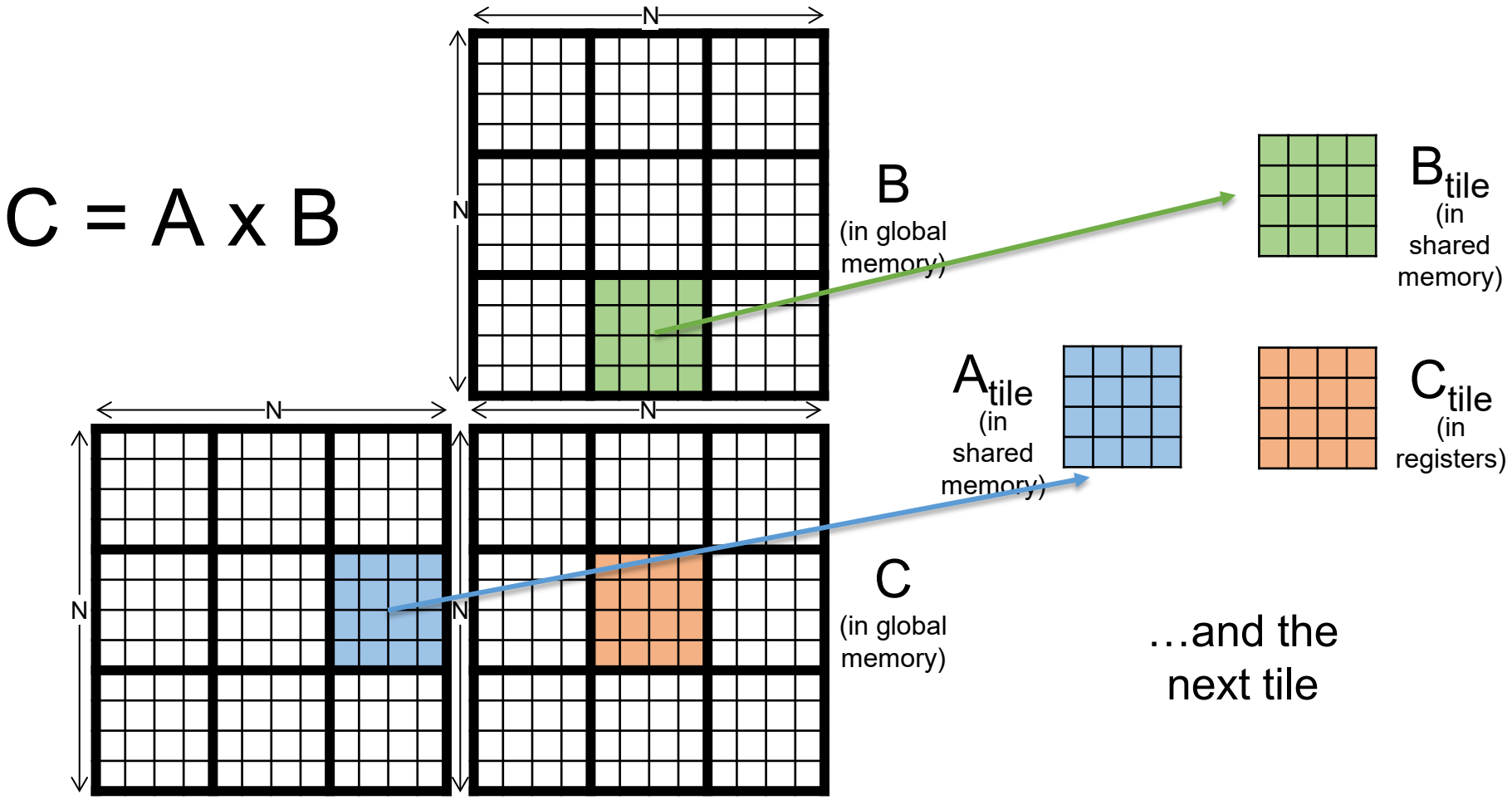


Step 2: Each thread computes its partial sum from the tiles in shared memory (threads wait for each other to finish)

Tiled Matrix-Matrix Multiplication



Tiled Matrix-Matrix Multiplication



Phase 1: Loading a Tile

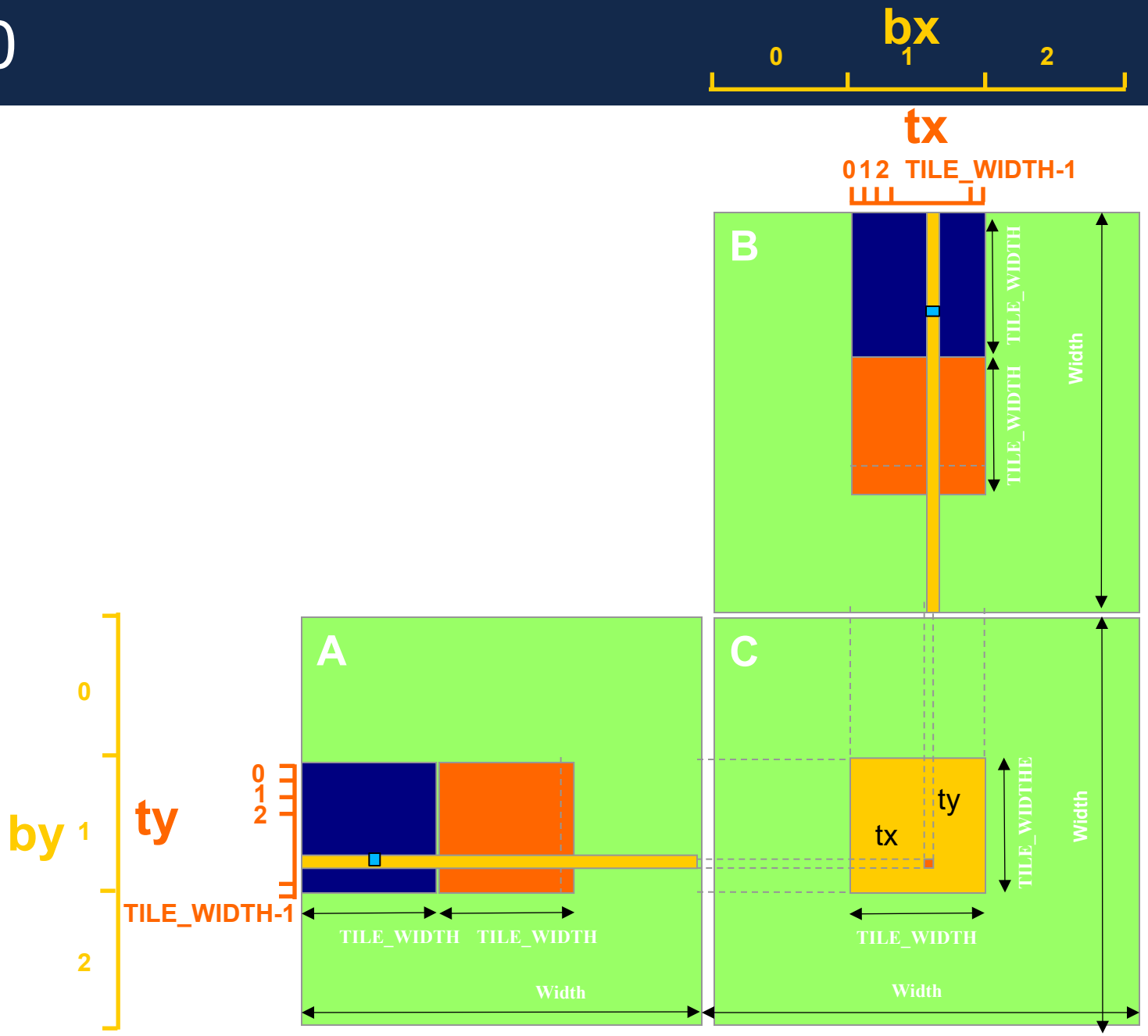
- All threads in a block participate
  - Each thread loads one A element and one B element in basic tiling code

- Assign the loaded element to each thread such that the accesses within each warp is coalesced (more later).

Loading an Input Tile 0

- 2D indexing for Tile 0

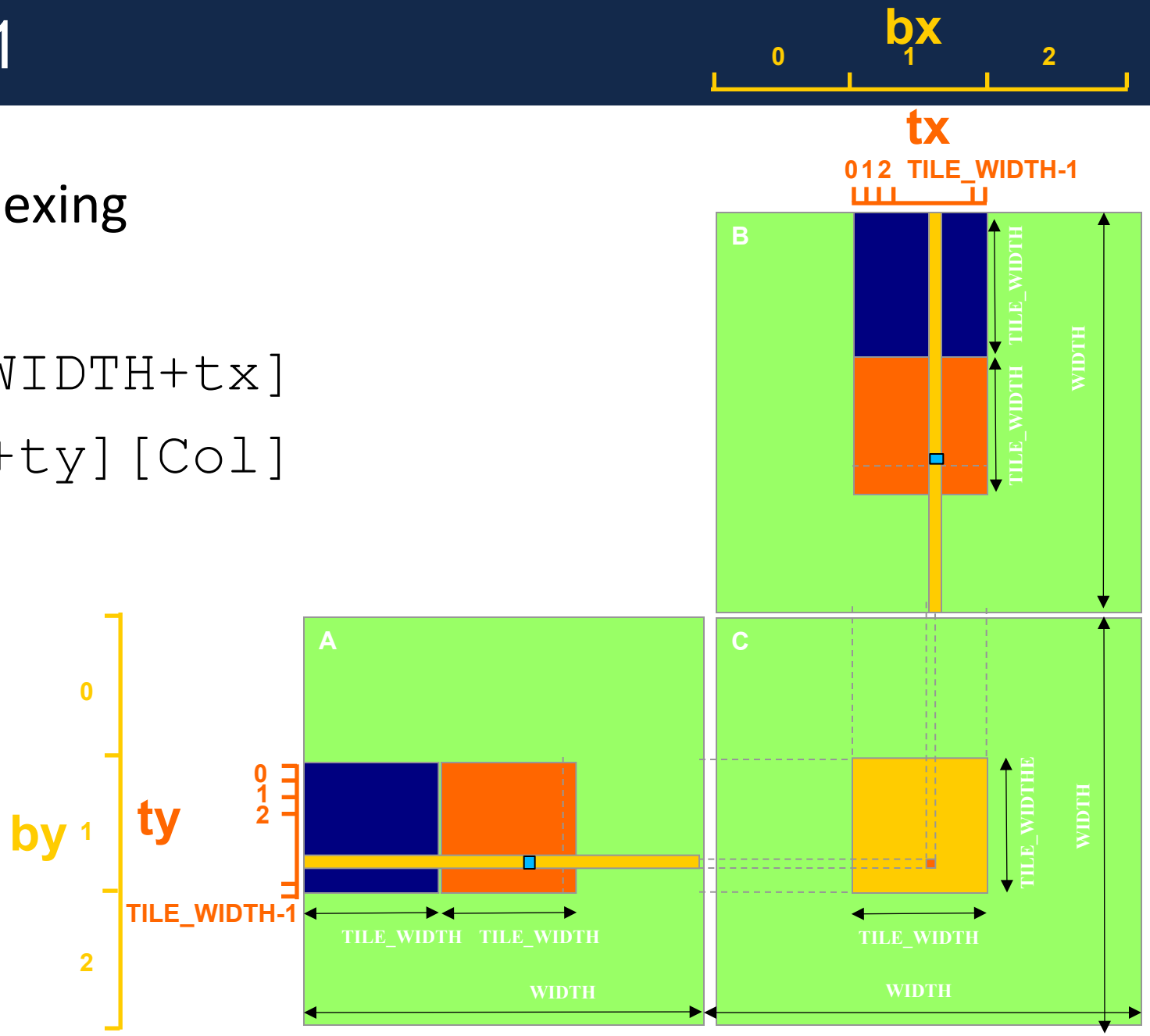
```
A[Row][tx]
B[ty][Col]
```



Loading an Input Tile 1

- Accessing tile 1 in 2D indexing

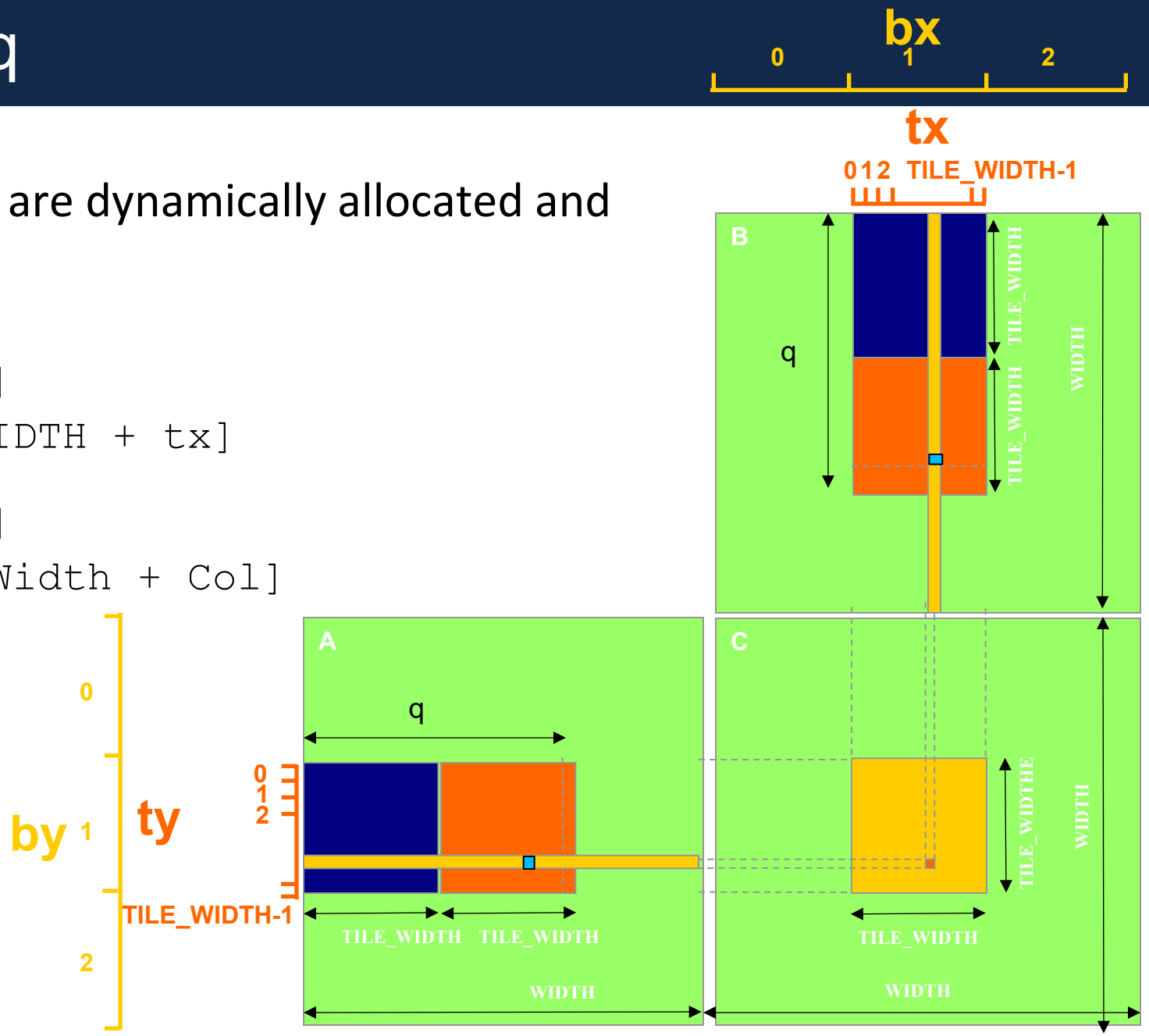
```
A[Row][1*TILE_WIDTH+tx]
B[1*TILE_WIDTH+ty][Col]
```



Loading an Input Tile q

- However, recall that A and B are dynamically allocated and can only use 1D indexing

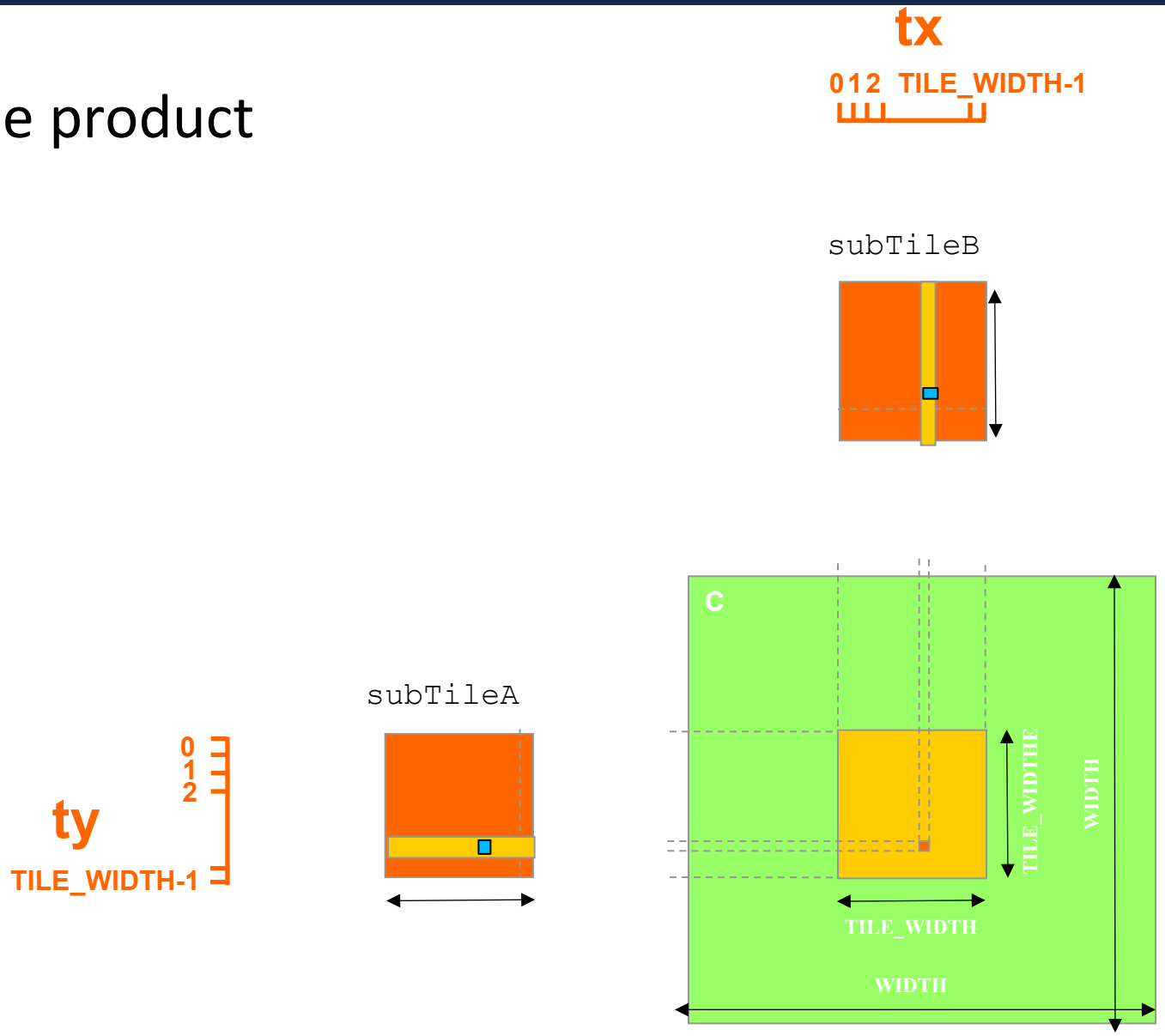
```
A[Row][q*TILE_WIDTH+tx]
A[Row*Width + q*TILE_WIDTH + tx]

B[q*TILE_WIDTH+ty][Col]
B[(q*TILE_WIDTH+ty) * Width + Col]
```



Phase 2: Compute partial product

- To perform the kth step of the product within the tile

```
subTileA[ty][k]
subTileB[k][tx]
```



Tiled Matrix-Matrix Multiplication

```
__shared__ float A_s[TILE_DIM][TILE_DIM];
__shared__ float B_s[TILE_DIM][TILE_DIM];

unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

float sum = 0.0f;

for (unsigned int tile = 0; tile < N/TILE_DIM; ++tile) {

    // Load tile to shared memory
    A_s[threadIdx.y][threadIdx.x] = A[row*N + tile*TILE_DIM + threadIdx.x];
    B_s[threadIdx.y][threadIdx.x] = B[(tile*TILE_DIM + threadIdx.y)*N + col];


    // Compute with tile
    for (unsigned int i = 0; i < TILE_DIM; ++i) {
        sum += A_s[threadIdx.y][i] * B_s[i][threadIdx.x];
    }


}

C[row*N + col] = sum;
```

Declare arrays in shared memory

We're Not There Yet!

- But …

- **How can a thread know** …
  - **That another thread has finished** its part of the tile?
  - Or that another thread has finished using the previous tile?

We need to synchronize!

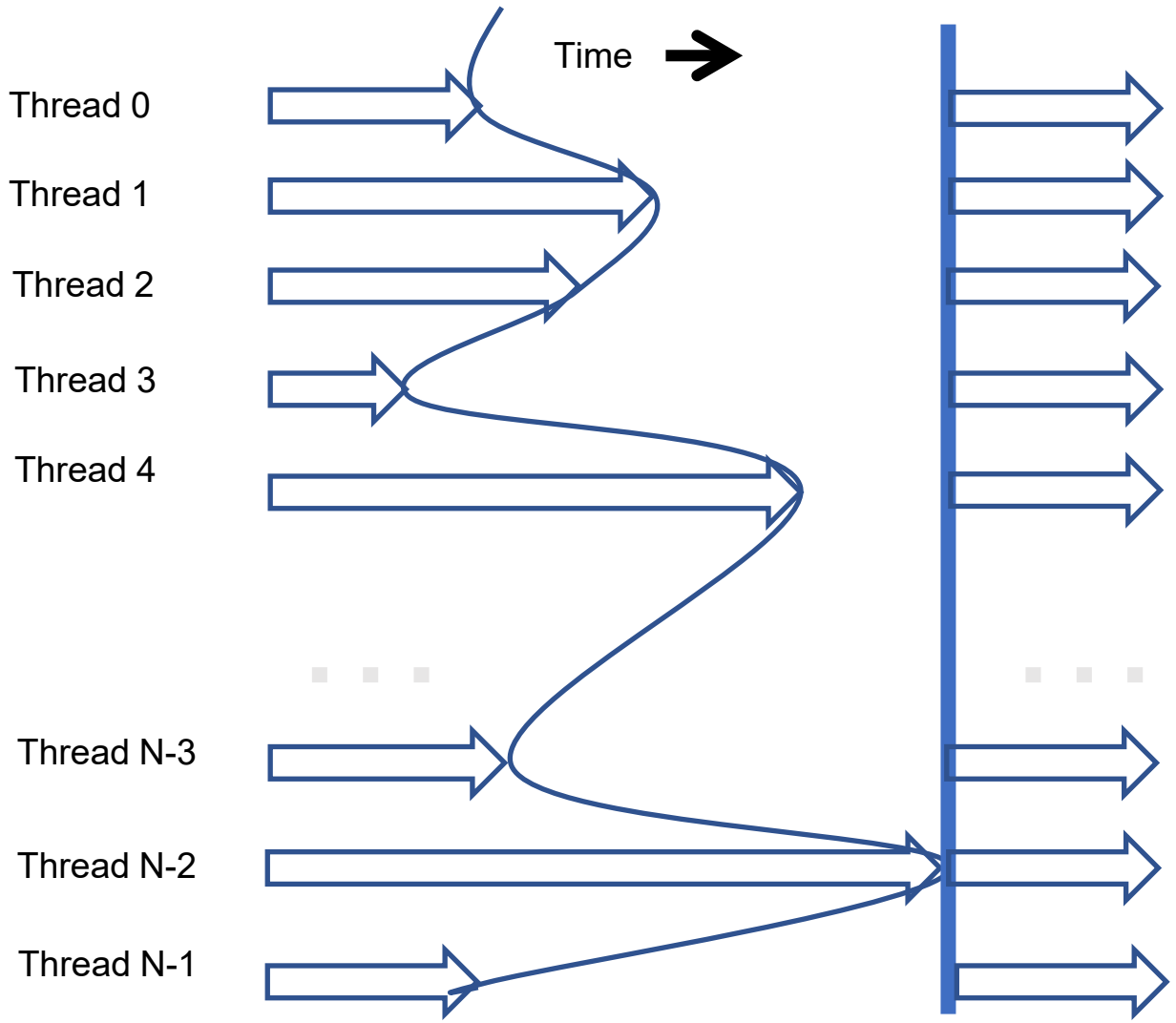
Leveraging Parallel Strategies

- **Bulk synchronous execution**: threads execute roughly in unison
  1. Do some work
  2. Wait for others to catch up
  3. Repeat
- **Much easier programming model**
  - Threads only parallel within a section
  - Debug lots of little programs
  - Instead of one large one
- **Dominates high-performance applications**

Bulk Synchronous Steps Based on Barriers

- **How does it work?**
  Use a barrier to wait for thread to 'catch up.'

- A barrier is a synchronization point:
  - **each thread calls a function** to enter barrier;
  - **threads block** (sleep) in barrier function
    until all threads have called;
  - **after last thread calls** function, **all threads continue** past the barrier.



Barrier Synchronization

- An API function call in CUDA `__syncthreads()`
- All threads **in the same block** must reach the `__syncthreads()` before any can move on
- Can be used to coordinate tiled algorithms
  - To ensure that all elements of a tile are loaded
  - To ensure that certain computation on elements is complete

Tiled Matrix-Matrix Multiplication

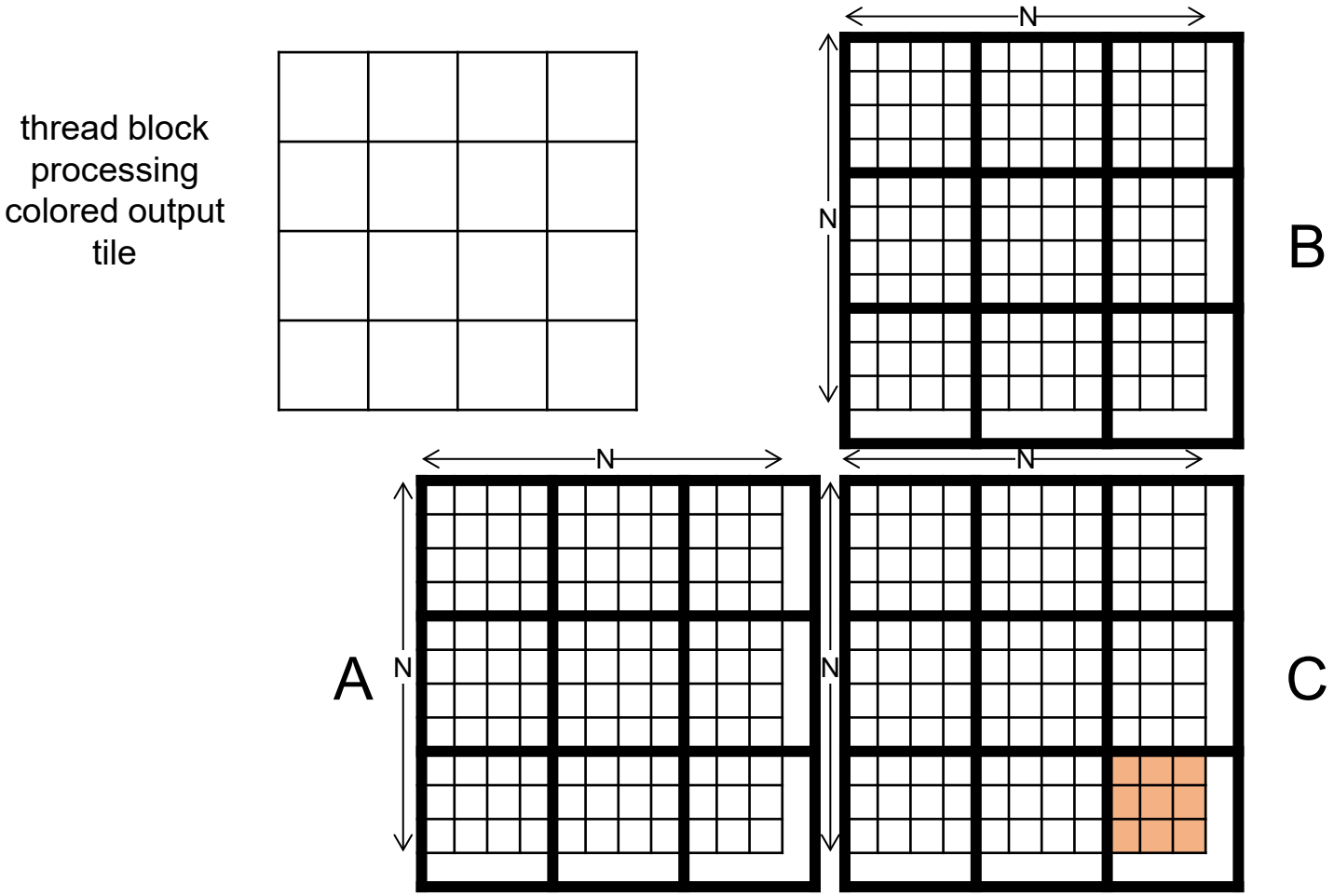
```
__shared__ float A_s[TILE_DIM][TILE_DIM];
__shared__ float B_s[TILE_DIM][TILE_DIM];

unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

float sum = 0.0f;

for (unsigned int tile = 0; tile < N/TILE_DIM; ++tile) {

    // Load tile to shared memory
    A_s[threadIdx.y][threadIdx.x] = A[row*N + tile*TILE_DIM + threadIdx.x];
    B_s[threadIdx.y][threadIdx.x] = B[(tile*TILE_DIM + threadIdx.y)*N + col];
    __syncthreads();

    // Compute with tile
    for (unsigned int i = 0; i < TILE_DIM; ++i) {
        sum += A_s[threadIdx.y][i] * B_s[i][threadIdx.x];
    }
    __syncthreads();

}

C[row*N + col] = sum;
```

Declare arrays in shared memory

Threads wait for each other to finish loading before computing

Threads wait for each other to finish computing before loading

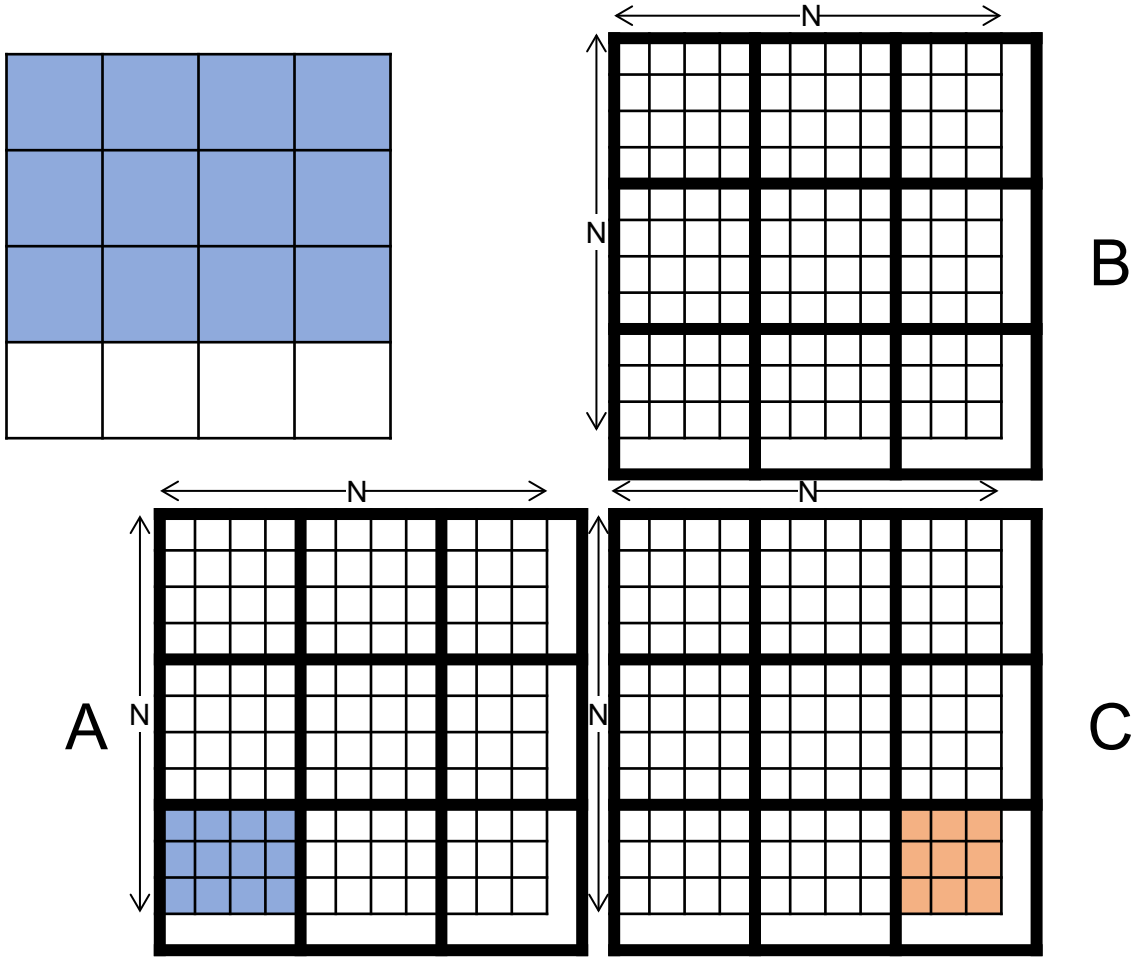
Boundary Conditions



Boundary Conditions

thread block processing colored output tile

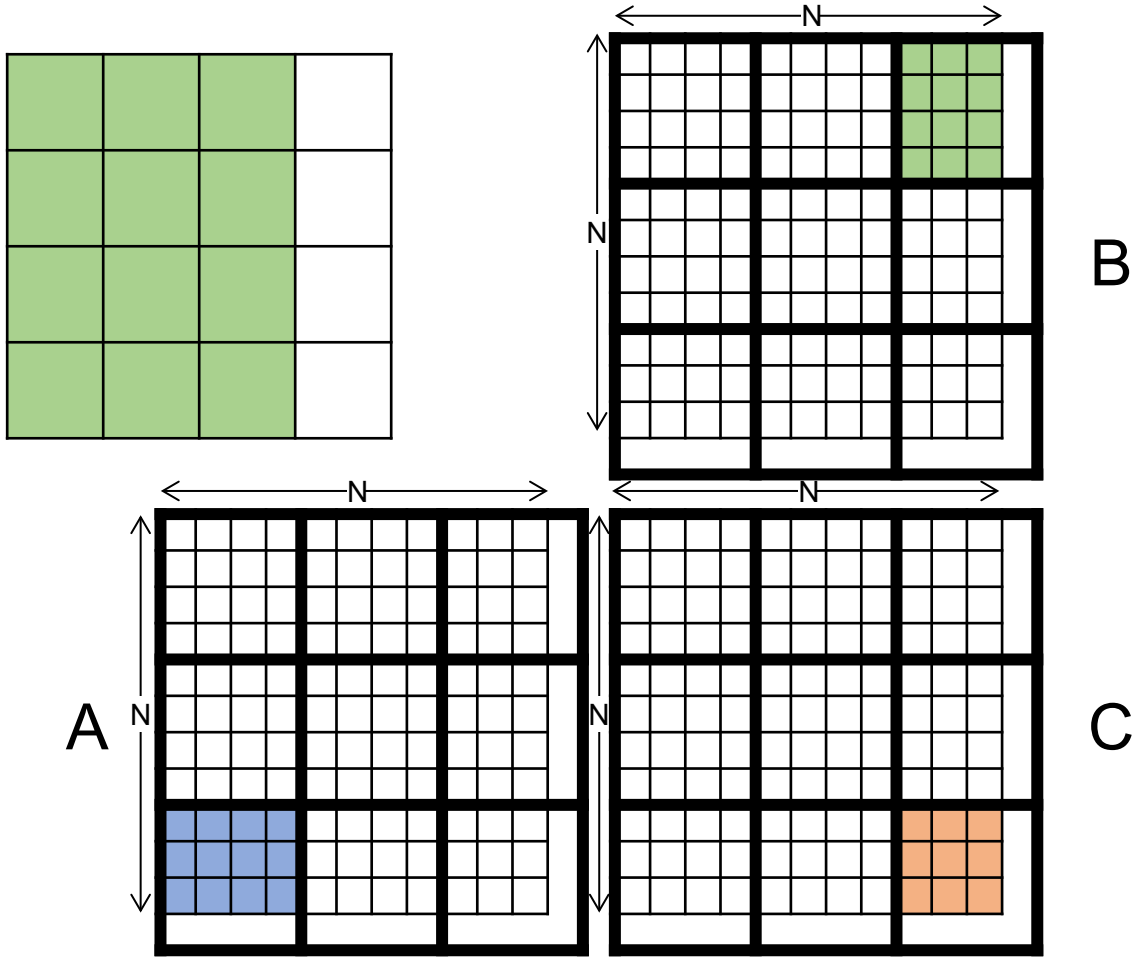
threads active while loading A tile



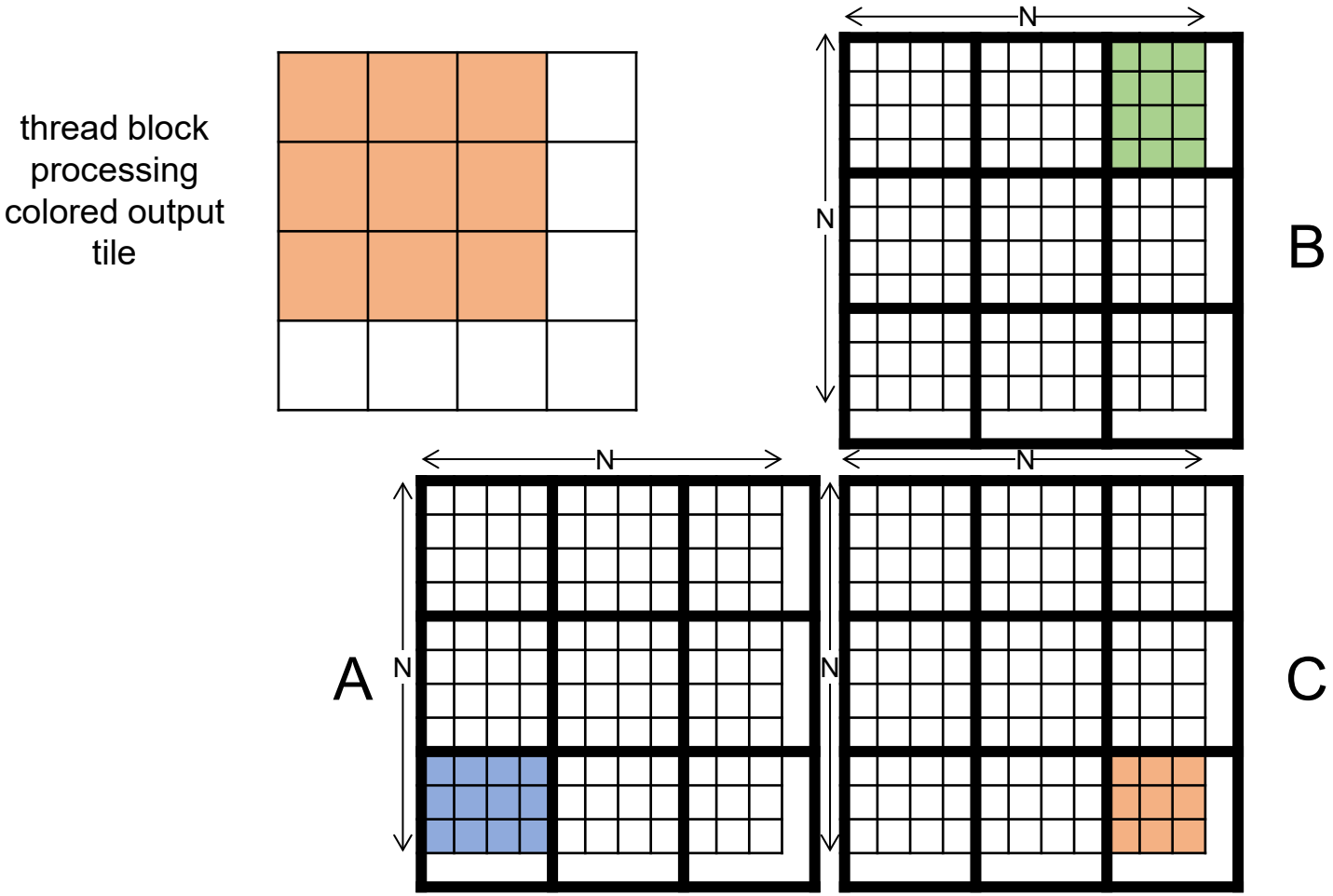
Boundary Conditions

thread block processing colored output tile

threads active while loading B tile

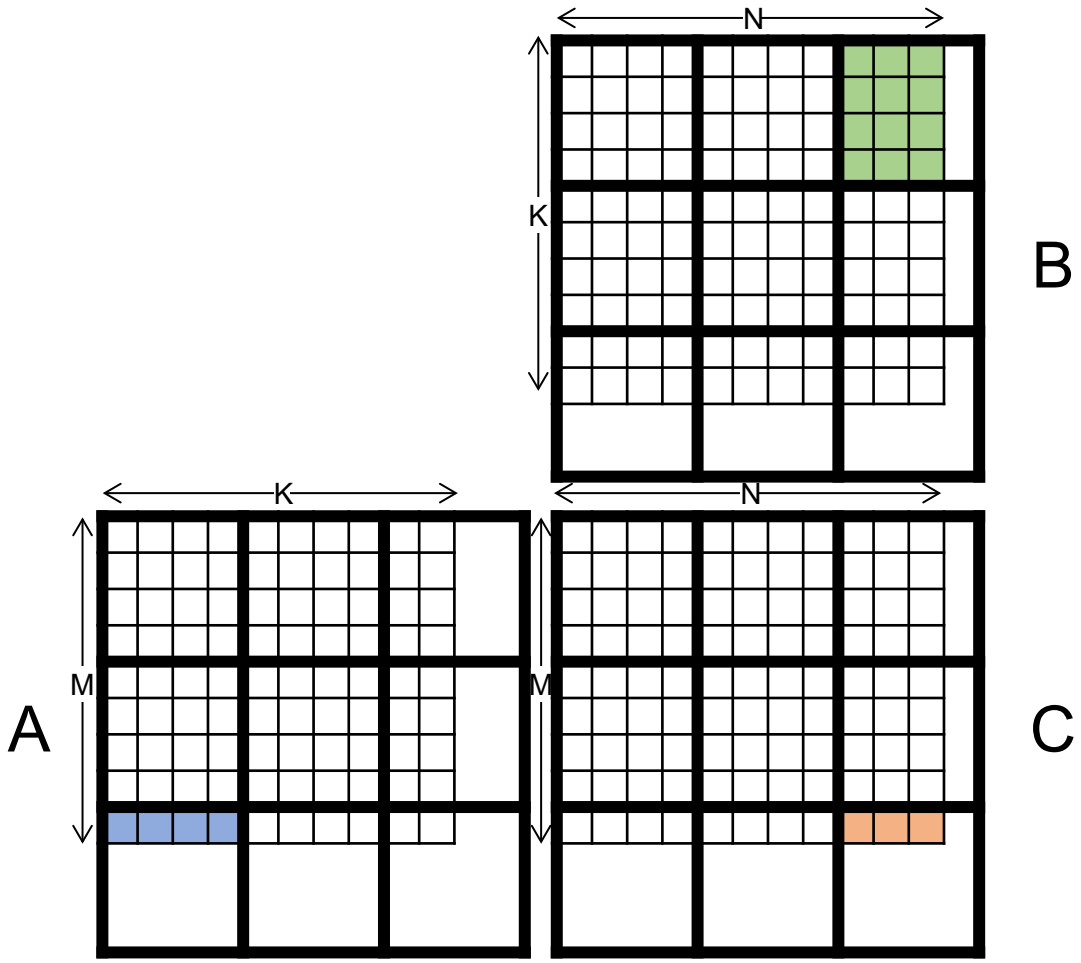


Boundary Conditions

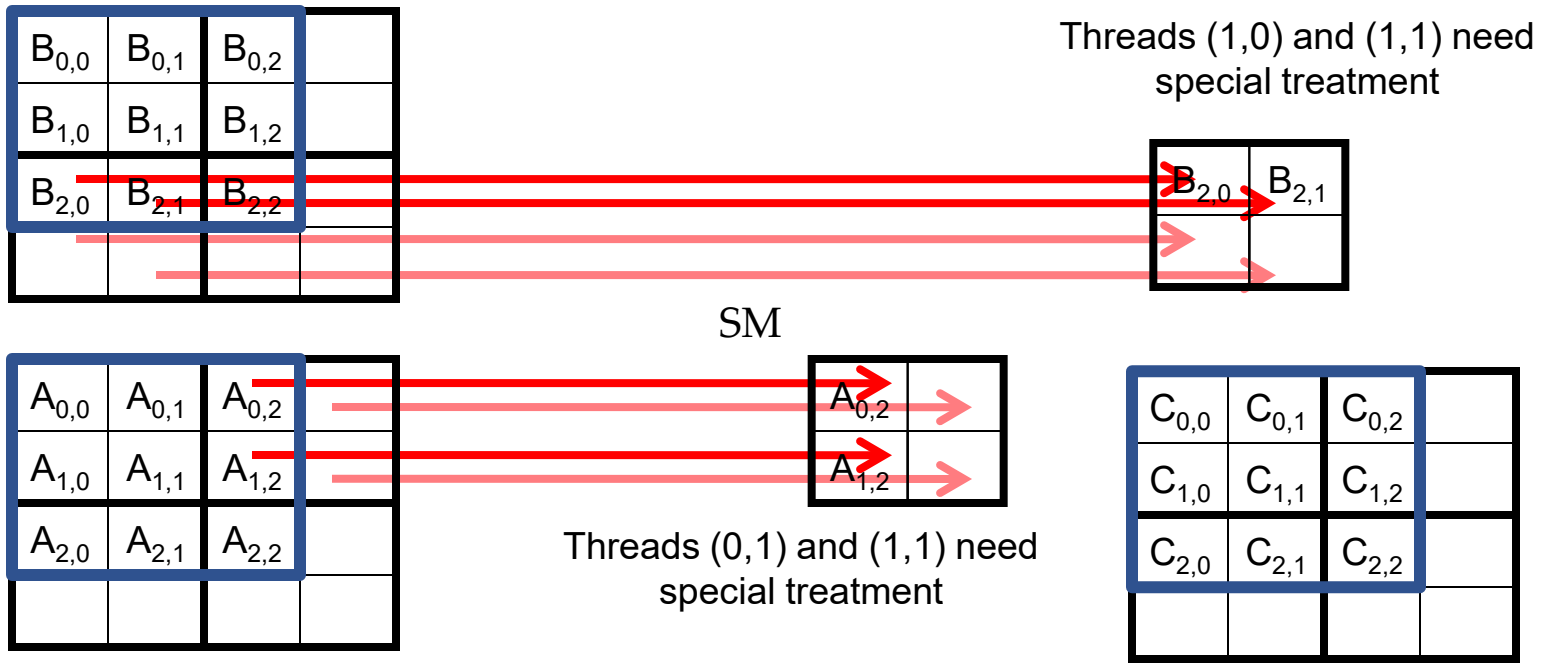


Different boundary conditions for different accesses (if a thread does not compute an output, this does not mean it is not needed for loading an input)

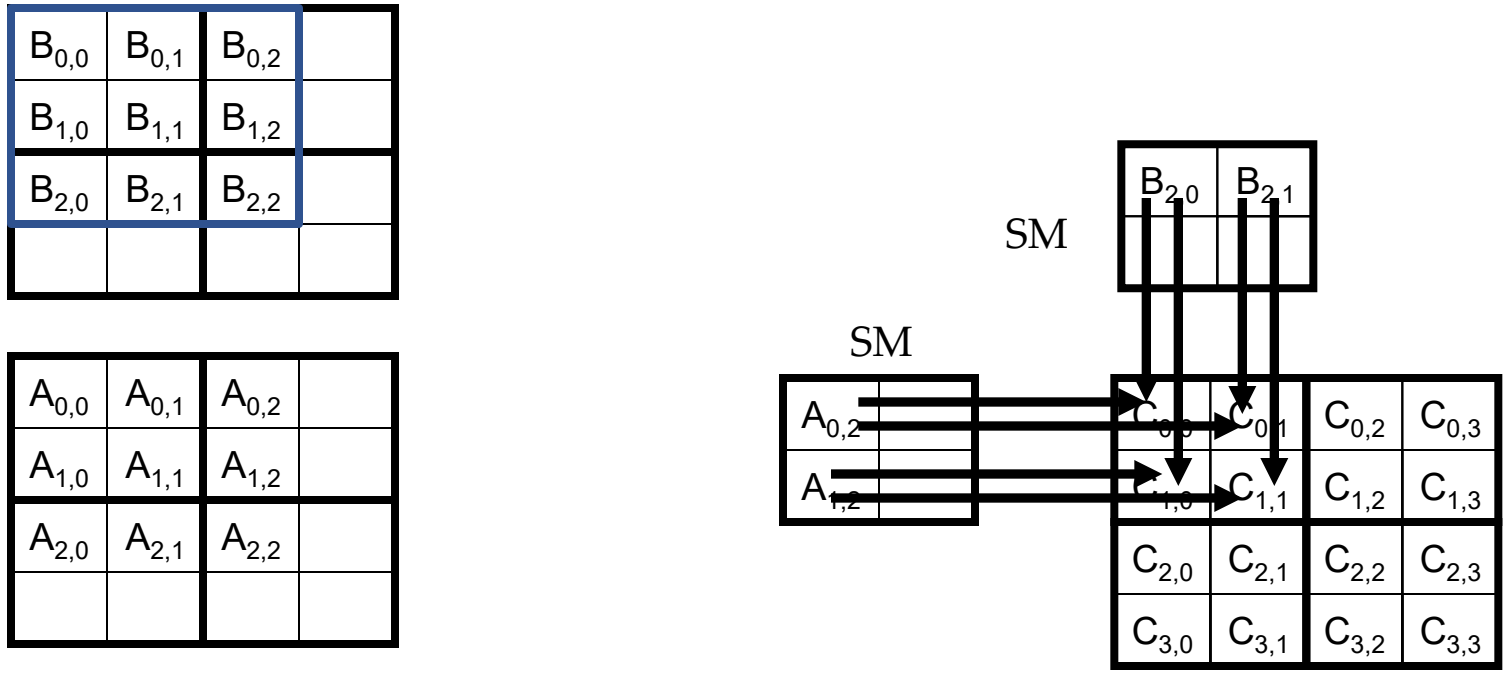
Different Matrix Dimensions



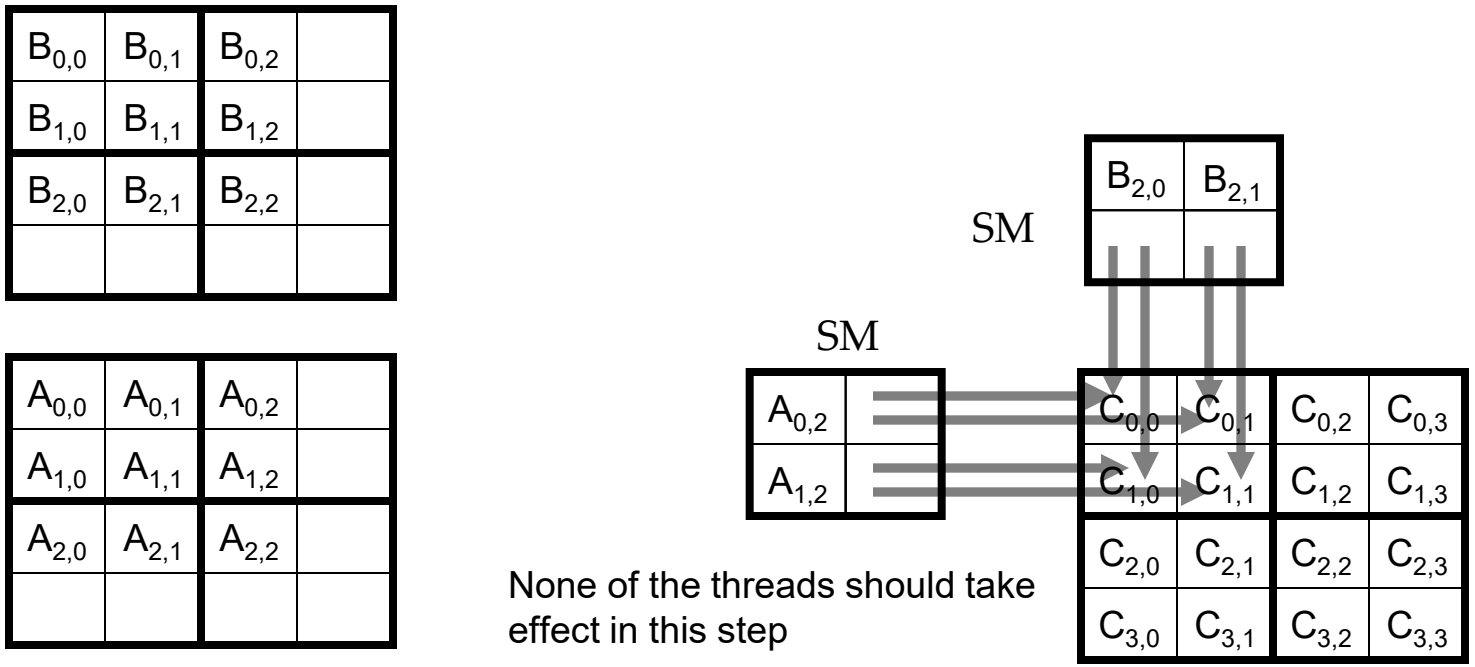
Example: Second Tile Load for Block (0,0)



Example: Second Tile Use for Block (0,0), k of 0



Example: Second Tile Use for Block (0,0), k of 1



None of the threads should take effect in this step

Major Cases in Toy Example

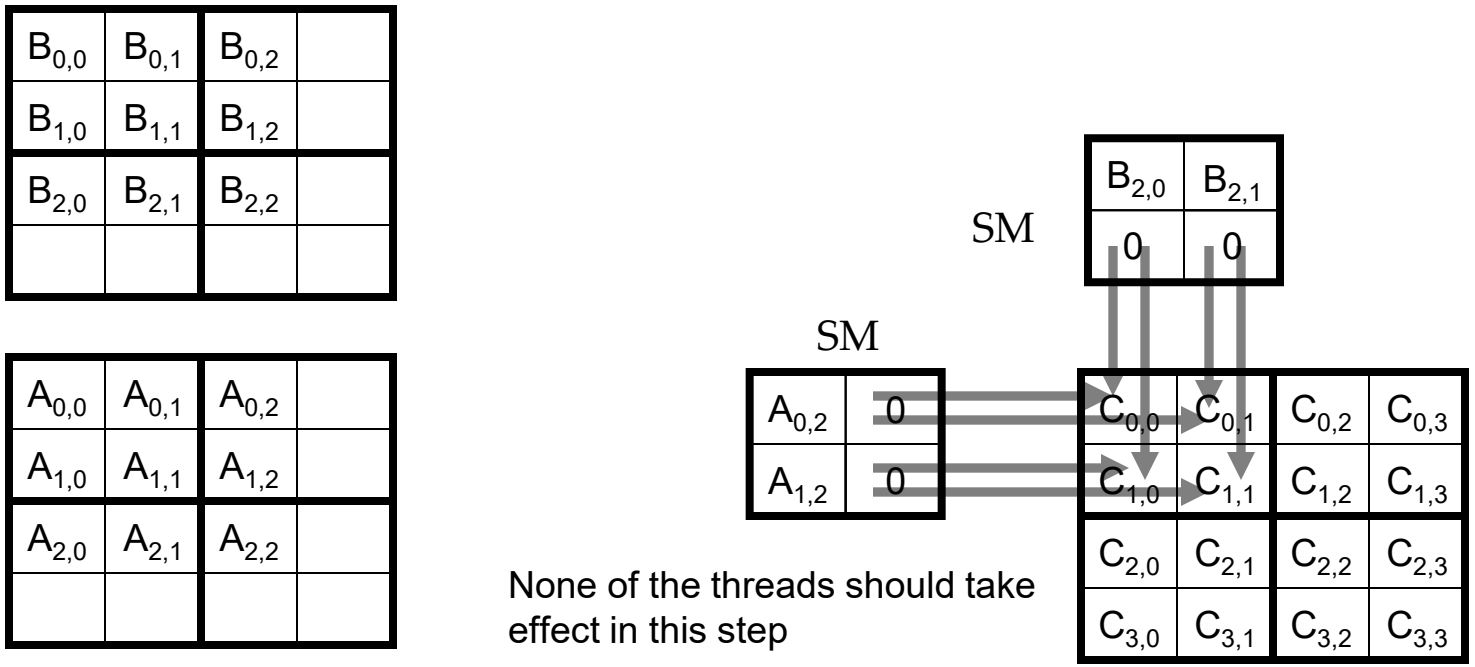
- Threads that calculate valid C elements but can step outside valid input
  - Second tile of Block(0,0), all threads when k is 1
- Threads that do not calculate valid C elements
  - Block(1,1), Thread(1,0), non-existent row
  - Block(1,1), Thread(0,1), non-existing column
  - Block(1,1), Thread(1,1), non-existing row/column

Solution: Write 0 for Missing Elements

- Test during tile load:
  is **target within input matrix**?
  - **If yes**, proceed to **load**;
  - **otherwise**, just **write 0** to shared memory.

- The **benefit**?
  - **No specialization during tile use**!
  - Multiplying by 0 guarantees that unwanted terms do not contribute to the inner product.

Example: Second Tile Use for Block (0,0), k of 1



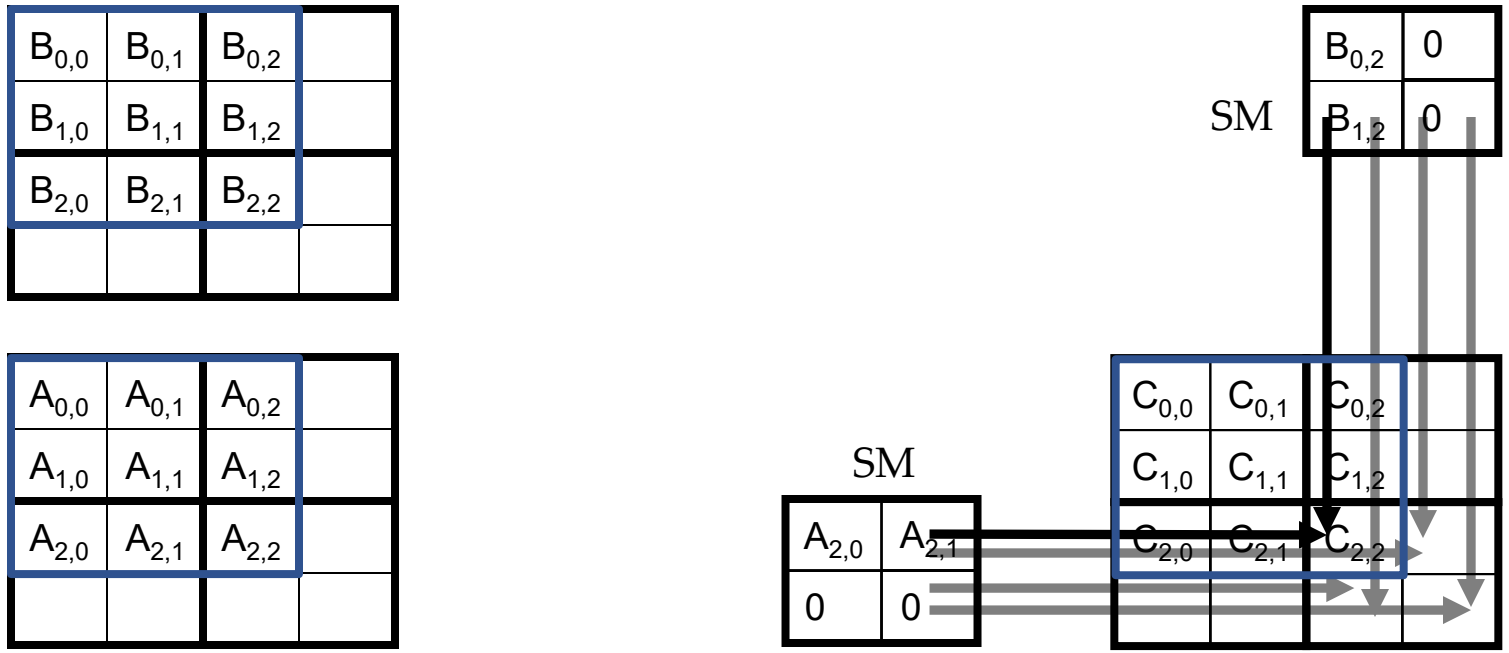
None of the threads should take effect in this step

What About Threads Outside of P?

- If a **thread is not within C**,
  - All terms in sum are 0.
  - No harm in performing FLOPs.
  - No harm in writing to registers.
  - **Must not be allowed to write to global memory!**

So: **Threads outside of C calculate 0, but store nothing.**

Example: First Tile Use for Block (1,1)



Modifying the Tile Count

```
for (unsigned int tile = 0; tile < N/TILE_DIM; ++tile) {
```

The bound for **`tile`** implicitly assumes that **N** is a multiple of **TILE_WIDTH**. We need to round up.

```
for (unsigned int tile = 0; tile < (N - 1)/TILE_DIM + 1; ++tile) {
```

For non-multiples of **`TILE_DIM`**:

- quotient is unchanged;
- add one to round up.

For multiples of **`TILE_DIM`**:

- quotient is now one smaller,
- but we add 1.

Modifying the Tile Loading Code

We had …

```
// Load tile to shared memory
A_s[threadIdx.y][threadIdx.x] = A[row*N + tile*TILE_DIM + threadIdx.x];
B_s[threadIdx.y][threadIdx.x] = B[(tile*TILE_DIM + threadIdx.y)*N + col];
```

Note: the tests for A and B tiles are NOT the same.

```
if (row < N && tile*TILE_DIM+threadIdx.x < N) {
    // as before
    A_s[threadIdx.y][threadIdx.x] = A[row*N + tile*TILE_DIM + threadIdx.x];
} else {
    A_s[threadIdx.y][threadIdx.x] = 0;
}
```

And for Loading B…

We had …

```
// Load tile to shared memory
A_s[threadIdx.y][threadIdx.x] = A[row*N + tile*TILE_DIM + threadIdx.x];
B_s[threadIdx.y][threadIdx.x] = B[(tile*TILE_DIM + threadIdx.y)*N + col];
```

Note: the tests for A and B tiles are NOT the same.

```
if (tile*TILE_DIM+threadIdx.y < N && col < N ) {
    // as before
    B_s[threadIdx.y][threadIdx.x] = B[(tile*TILE_DIM + threadIdx.y)*N + col];
} else {
    B_s[threadIdx.y][threadIdx.x] = 0;
}
```

Modifying the Tile Use Code

We had …

```
    // Compute with tile
    for (unsigned int i = 0; i < TILE_DIM; ++i) {
        sum += A_s[threadIdx.y][i] * B_s[i][threadIdx.x];
    }
```

Note: **no changes are needed**, but we might save a little energy (fewer floating-point ops)?

```
if (row < N && col < N) {
    // as before
    for (unsigned int i = 0; i < TILE_DIM; ++i) {
        sum += A_s[threadIdx.y][i] * B_s[i][threadIdx.x];
}
```

Modifying the Write to C

We had …

```
C[row*N + col] = sum;
```

We must test for threads outside of C:

```
if (row < N && col < N) {
    // as before
    C[row*N + col] = sum;
}
```

Some Important Points

- For each thread, conditions are different for
  - Loading A element
  - Loading B element
  - Calculation/storing output elements
- Branch divergence
  - affects only blocks on boundaries, and
  - should be small for large matrices.

Use of Large Tiles Shifts Bottleneck

- Recall our example GPU: **1,000 GFLOP/s**, **150 GB/s**
- **16x16 tiles** use each operand for 16 operations
  - **reduce global** memory **accesses by** a factor of **16**
  - **150GB/s** bandwidth supports (150/4)*16 = **600 GFLOPS**!
- **32x32 tiles** use each operand for 32 operations
  - **reduce global** memory **accesses by** a factor of **32**
  - **150 GB/s** bandwidth supports (150/4)*32 = **1,200 GFLOPS**!
  - **Memory bandwidth is no longer the bottleneck!**

Also Need Parallel Accesses to Memory

- Shared memory size
  - implementation dependent
  - **64kB** per SM in Maxwell (48kB max per block)
- Given **TILE_WIDTH of 16** (256 threads / block),
  - each thread block uses 2*256*4B = 2kB of shared memory,
  - which limits active blocks to 32;
  - max. of 2048 threads per SM,
  - which limits blocks to 8.
  - Thus, up to 8*512 = **4,096 pending loads**
    (2 per thread, 256 threads per block)

Another Good Choice: 32x32 Tiles

- Given **TILE_WIDTH of 32** (1,024 threads / block),
  - each thread block uses 2*1024*4B = 8kB of shared memory,
  - which limits active blocks to 8;
  - max. of 2,048 threads per SM,
  - which limits blocks to 2.
  - Thus, up to 2*2,048 = **4,096 pending loads**
    (2 per thread, 1,024 threads per block)

(same memory parallelism exposed)

Memory and Occupancy

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Compute Capability	3.5	5.2	6.0	7.0	8.0	9.0
Number of SMs	15	24	56	80	108	132
Cores per SM	192	128	64	64	64	128
Max threads per SM	2048	2048	2048	2048	2048	2048
Max blocks per SM	16	32	32	32	32	32
Max threads per block	1024	1024	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB	164KB	228KB

Register usage per thread, and shared memory usage per thread block constrain occupancy

Dynamic Shared Memory

- Shared memory can be dynamically allocated if programmer does not know how much is needed beforehand

- Declaration:
  - `extern __shared__ A_s[];`

- Configuration:
  - `kernel <<< numBlocks, numThreadsPerBlock, smemPerBlock >>> (...)`

Advanced Features

- Starting with the Hopper architecture:
  - **Distributed shared memory** allows thread blocks in the same *thread block cluster* to accesses each others' shared memory
  - **Tensor memory accelerator (TMA)** allows asynchronous transfer of data between global memory and shared memory
    - Covered later

Tiling on CPU

- Tiling also works for CPU
  - No scratchpad memory, but relies on caches
  - Cache is sufficiently reliable because there are fewer threads running on the core and the cache is larger

```
for (unsigned int rowTile = 0; rowTile < N/TILE_DIM; ++rowTile) {
        for (unsigned int colTile = 0; colTile < N/TILE_DIM; ++colTile) {
            for (unsigned int iTile = 0; iTile < N/TILE_DIM; ++iTile) {
                for (unsigned int row = rowTile*TILE_DIM; row < (rowTile + 1)*TILE_DIM; ++row) {
                    for (unsigned int col = colTile*TILE_DIM; col < (colTile + 1)*TILE_DIM; ++col) {
                        float sum = 0.0f;
                        for (unsigned int i = iTile*TILE_DIM; i < (iTile + 1)*TILE_DIM; ++i) {
                            sum += A[row*N + i]*B[i*N + col];
                        }
                        if (iTile == 0) C[row*N + col] = sum;
                        else  C[row*N + col] += sum;
                }
            }
        }
    }
```

Any More Questions?

Things to Read:

Textbook chapter 5

CUDA PG: Programming Model

Things to Do:

Get Delta account

Setup GitHub

Submit Labs 0, 1, and 2